

C++11 Language Cheat Sheet

A clean study book for modern C++ foundations

```
auto v = std::vector<int>{1, 2, 3};  
for (const auto& x : v) { /* ... */ }  
auto f = [](int x) { return x * x; };  
std::unique_ptr<T> p(new T{});  
template<class T> void use(T&& value);
```

C++11 Language Cheat Sheet Study Book

This booklet is designed as a compact but serious study material. It explains the vocabulary behind C++11, gives syntax patterns, shows edge cases, and connects individual features to practical programming habits.

How to use it

- Use the vocabulary boxes to learn the language used by C++ documentation and compiler diagnostics.
- Copy the examples into a small compiler project and compile with `-std=c++11 -Wall -Wextra`.
- When a feature has a trap, read the edge case before memorizing the syntax.
- Prefer understanding ownership, lifetime, value categories, and type deduction before chasing advanced templates.

Recommended compiler flag

```
g++ -std=c++11 -Wall -Wextra -pedantic main.cpp -o app
```

Table of contents

How to use it	2
Table of contents	3
1. What C++11 Changed	7
Core vocabulary	7
2. Minimal Program, Build Model, and Namespaces	7
Minimal structure	7
Common file roles	8
Namespaces	8
3. Types, Variables, and Initialization	8
Fundamental type categories	8
Initialization forms	8
Narrowing edge case	8
const and constexpr	8
4. Expressions, Operators, and Control Flow	9
Operator reference	9
if, switch, loops	9
Range-based for	9
5. Functions and Parameter Passing	9
Function anatomy	9
Passing parameters	10
Default arguments and overloads	10
Trailing return type	10
6. auto, decltype, and Type Deduction	10
auto	10
Important auto edge cases	10
decltype	11

7. Pointers, References, and nullptr	11
Pointers and references	11
nullptr	11
Dangling reference danger	11
8. Arrays, Strings, and Containers	11
Built-in arrays vs std::array vs std::vector	11
std::string essentials	12
Container cheat sheet	12
9. Classes, Structs, Constructors, and RAI	12
class vs struct	12
Class skeleton	12
Initializer lists	12
RAII	13
10. Special Member Functions and the Rule of Five	13
The special members	13
= default and = delete	13
Rule of zero, three, five	13
11. Move Semantics and Value Categories	14
Why move exists	14
lvalues and rvalues	14
std::move	14
Perfect forwarding overview	14
12. Smart Pointers and Ownership	14
Ownership vocabulary	14
C++11 smart pointer summary	15
shared_ptr cycle edge case	15
13. Lambdas, Function Objects, and Callbacks	15
Lambda syntax	15

Capture modes	15
Lambda with STL algorithm	15
std::function	16
14. Templates and Generic Programming	16
Function template	16
Class template	16
Alias templates	16
Variadic templates	16
15. Inheritance, Polymorphism, override, and final	17
Basic inheritance	17
override and final	17
16. enum class and Stronger Type Safety	17
Old enum vs enum class	17
Underlying type	17
17. STL Algorithms and Iterators	18
Iterator vocabulary	18
Common algorithms	18
Iterator invalidation edge case	18
18. Exceptions and Error Handling	18
try, throw, catch	18
noexcept	19
Exception safety levels	19
19. Concurrency in C++11	19
std::thread	19
mutex and lock_guard	19
future and async	19
atomic	19
20. Chrono, Random, Tuple, Regex, and Utility Types	20

chrono	20
random	20
tuple and pair	20
regex	20
21. Preprocessor, Headers, and Compilation Hygiene	20
Include guards	20
Macros	21
Forward declarations	21
22. C++11 Cheat Sheets	21
Keywords and feature reminders	21
STL headers quick reference	21
Common compiler diagnostics vocabulary	22
23. Pitfalls and Edge Cases	22
Most common C++11 mistakes	22
Undefined behavior examples	22
24. Mini Practice Lab	23
Exercise 1: Replace raw ownership	23
Exercise 2: Use algorithms	23
Exercise 3: Write a safe callback	23
Exercise 4: Thread safety	23
25. Compact Glossary	23
26. Final One-Page Review	24

1. What C++11 Changed

C++11 was the first major modernization of C++ after C++98/03. It kept compatibility with classic C++, but added language and library tools that make code safer, clearer, and more expressive.

Area	C++11 idea	Why it matters
Type deduction	auto, decltype	Less repetition; easier generic code.
Resource management	move semantics, smart pointers	Avoid needless copies and make ownership explicit.
Functions as values	lambdas, std::function, bind	Cleaner callbacks and algorithms.
Initialization	uniform initialization, initializer_list	More consistent object creation.
Concurrency	thread, mutex, future, atomic	Portable standard threading.
Templates	variadic templates, alias templates	More flexible generic libraries.
Library	unordered containers, tuple, chrono, regex	Common tools standardized.

A useful mental model: C++11 is still C++, so object lifetime, copying, references, pointers, const-correctness, and undefined behavior still matter. The new features help you express intent, but they do not remove the need to understand the underlying rules.

Core vocabulary

Term	Meaning
object	A region of storage with a type and lifetime.
value	The actual data represented by an expression or object.
lifetime	The period during which an object exists and can be used.
scope	The source-code region where a name can be used.
RAII	Resource Acquisition Is Initialization: tie resource lifetime to object lifetime.
undefined behavior	The program has no required meaning; anything may happen.
implementation-defined	The compiler/platform must document what it does.
unspecified	Several valid choices exist; the implementation need not document which one.

2. Minimal Program, Build Model, and Namespaces

Minimal structure

```
#include <iostream>

int main()
{
    std::cout << "Hello, C++11\n";
    return 0;
}
```

A C++ program is usually compiled translation unit by translation unit. Headers are textually included by the preprocessor. Source files are compiled into object files, then linked into the final executable.

Common file roles

File	Typical content	Rule of thumb
.h / .hpp	Declarations, class definitions, templates, inline functions	Include guards or #pragma once.
.cpp	Function definitions, non-template implementation	Compile this file.
.o / .obj	Compiled object file	Produced by compiler, consumed by linker.
library	Reusable compiled code	Static or shared/dynamic.

Namespaces

```
namespace device {
    void start();
}

device::start();
```

Namespaces prevent name collisions. Avoid putting `using namespace std;` in headers because it leaks names into every file that includes the header.

3. Types, Variables, and Initialization

Fundamental type categories

Category	Examples	Notes
Boolean	bool	Stores true or false.
Character	char, wchar_t, char16_t, char32_t	C++11 added char16_t and char32_t.
Integer	short, int, long, long long	Signed overflow is undefined behavior.
Floating point	float, double, long double	Approximate real numbers, not exact decimals.
Void	void	No value; used for functions returning nothing and generic pointers.

Initialization forms

```
int a = 10;      // copy initialization
int b(10);      // direct initialization
int c{10};      // brace initialization, preferred in many C++11 contexts
int d = {10};   // copy-list initialization

std::vector<int> v{1, 2, 3};
```

Brace initialization is important in C++11 because it prevents many narrowing conversions and works for aggregates, classes, and standard containers.

Narrowing edge case

```
int x = 3.14;    // allowed, x becomes 3, usually warns
int y{3.14};    // error: narrowing conversion
```

Use braces when you want the compiler to protect you from accidental loss of information.

const and constexpr


```
const int runtime_value = std::rand();    // const, but not compile-time constant
constexpr int buffer_size = 128;         // compile-time constant expression

int buffer[buffer_size];
```

Keyword	Meaning	Typical use
const	Value cannot be modified through that name	Read-only variables, references, member functions.
constexpr	Can be evaluated at compile time when inputs allow	Array sizes, constants, compile-time functions.

4. Expressions, Operators, and Control Flow

Operator reference

Group	Operators	Common trap
Arithmetic	+ - * / %	Integer division truncates: 5 / 2 is 2.
Comparison	== != < <= > >=	Do not compare floating-point values for exact equality unless intended.
Logical	&& !	&& and short-circuit.
Bitwise	& ^ ~ << >>	Different from logical operators.
Assignment	= += -= *= /=	Assignment returns the assigned value as an expression.
Member access	. -> ::	Use -> through pointer, . through object/reference.

if, switch, loops

```
if (temperature > 80) {
    stop_motor();
} else {
    continue_running();
}

switch (state) {
    case State::Idle:    break;
    case State::Active: break;
    default:            break;
}

for (int i = 0; i < 10; ++i) { }
while (condition) { }
do { } while (condition);
```

Range-based for

```
std::vector<std::string> names{"Ada", "Bjarne", "Grace"};

for (const auto& name : names) {
    std::cout << name << '\n';
}
```

Use `const auto&` when you want to read elements without copying them. Use `auto&` to modify elements. Use plain `auto` when a copy is cheap or intended.

5. Functions and Parameter Passing

Function anatomy

```

return_type function_name(parameter_list)
{
    // body
}

int add(int a, int b)
{
    return a + b;
}

```

Passing parameters

Form	Example	Meaning
By value	<code>void f(std::string s)</code>	Function receives a copy or moved value.
By lvalue reference	<code>void f(std::string& s)</code>	Function can modify caller's object.
By const reference	<code>void f(const std::string& s)</code>	Read without copying.
By pointer	<code>void f(int* p)</code>	Can pass null; expresses optional object or C-style API.
By rvalue reference	<code>void f(std::string&& s)</code>	Can bind to temporary/movable object.

Default arguments and overloads

```

void log(const std::string& msg, int level = 1);

void print(int value);
void print(const std::string& value);

```

Overloading chooses a function based on argument types. Default arguments belong in declarations, usually in the header, not repeated in every definition.

Trailing return type

```

auto multiply(int a, int b) -> int
{
    return a * b;
}

```

Trailing return type is useful in templates where the return type depends on parameter expressions.

6. auto, decltype, and Type Deduction

auto

```

auto i = 42;           // int
auto d = 3.14;         // double
auto s = "text";       // const char*
auto v = std::vector<int>{1, 2, 3};

```

`auto` deduces a type from the initializer. It removes repetition, but it does not mean dynamic typing. The type is fixed at compile time.

Important auto edge cases

```
const int ci = 10;
auto a = ci;      // int, top-level const is dropped
auto& b = ci;     // const int&, reference preserves constness

std::vector<bool> flags{true, false};
auto x = flags[0]; // may be proxy object, not actual bool
```

When exact type matters, especially with references, proxies, or initializer lists, check deduction carefully.

decltype

```
int x = 0;
decltype(x) a = 1;    // int
decltype((x)) b = x;  // int&, because (x) is an lvalue expression
```

decltype asks the compiler for the type of an expression. Parentheses can change the result by changing expression category.

7. Pointers, References, and nullptr

Pointers and references

Concept	Syntax	Meaning
Pointer	int* p	Stores an address. Can be null. Can be reassigned.
Reference	int& r	Alias to an existing object. Cannot be resealed.
Const pointer	int* const p	The pointer itself cannot point elsewhere.
Pointer to const	const int* p	Cannot modify the pointed object through p.
Const reference	const int& r	Read-only alias; often used for parameters.

nullptr

```
void f(int);
void f(char*);

f(0);      // calls f(int)
f(NULL);   // may be ambiguous or call f(int), depending on definition
f(nullptr); // calls f(char*)
```

C++11 introduced nullptr as a real null pointer value. Prefer it over 0 or NULL.

Dangling reference danger

```
const std::string& bad()
{
    std::string local = "gone";
    return local; // wrong: local is destroyed when function returns
}
```

A reference does not own an object. Returning a reference or pointer to a local variable creates a dangling reference or dangling pointer.

8. Arrays, Strings, and Containers

Built-in arrays vs std::array vs std::vector

Type	Size	Use
T arr[N]	Fixed at compile time	Low-level interop; decays to pointer easily.

Type	Size	Use
<code>std::array<T,N></code>	Fixed at compile time	Safer fixed-size array with STL interface.
<code>std::vector<T></code>	Dynamic	Default choice for resizable sequences.

```
std::array<int, 3> fixed{{1, 2, 3}};    // double braces common in C++11
std::vector<int> dynamic{1, 2, 3};
dynamic.push_back(4);
```

std::string essentials

```
std::string name = "George";
name += " Calin";

if (name.find("Calin") != std::string::npos) {
    std::cout << name.substr(0, 6) << '\n';
}
```

`std::string::npos` means not found. Do not compare it to `-1` in new code; use the named constant.

Container cheat sheet

Container	Strength	C++11 note
vector	Fast indexed access, append at end	Use as default sequence container.
deque	Fast push/pop at both ends	Useful for queues.
list	Stable iterators, splice	Poor cache locality; not default.
map	Sorted key-value tree	$O(\log n)$.
unordered_map	Hash table	C++11 standardizes hash maps.
set	Sorted unique keys	$O(\log n)$.
unordered_set	Hash set	Average $O(1)$ lookup.

9. Classes, Structs, Constructors, and RAI

class vs struct

In C++, `class` and `struct` are nearly identical. The default access is `private` for `class` and `public` for `struct`. Use `struct` for simple data aggregates and `class` for types with invariants or behavior.

Class skeleton

```
class Device {
public:
    explicit Device(std::string id) : id_(std::move(id)) {}

    void start();
    const std::string& id() const { return id_; }

private:
    std::string id_;
};
```

Initializer lists

```

class Sensor {
public:
    Sensor(int pin, std::string name)
        : pin_(pin), name_(std::move(name)) // initialize members here
    {}

private:
    int pin_;
    std::string name_;
};

```

Prefer member initializer lists over assignment inside the constructor body. Some members, such as references, const members, and members without default constructors, must be initialized this way.

RAII

```

class FileHandle {
public:
    explicit FileHandle(FILE* f) : f_(f) {}
    ~FileHandle() { if (f_) std::fclose(f_); }

private:
    FILE* f_;
};

```

RAII means cleanup is automatic when an object leaves scope. This is the central C++ technique for memory, files, locks, sockets, and other resources.

10. Special Member Functions and the Rule of Five

The special members

Function	Purpose
Default constructor	Creates an object with no explicit arguments.
Destructor	Cleans up when lifetime ends.
Copy constructor	Creates a new object from an existing lvalue.
Copy assignment	Replaces an existing object from an lvalue.
Move constructor	Creates a new object by stealing resources from an rvalue.
Move assignment	Replaces an existing object by stealing resources from an rvalue.

= default and = delete

```

class NonCopyable {
public:
    NonCopyable() = default;
    NonCopyable(const NonCopyable&) = delete;
    NonCopyable& operator=(const NonCopyable&) = delete;
};

```

C++11 lets you explicitly request compiler-generated functions with `= default` and forbid operations with `= delete`.

Rule of zero, three, five

Rule	Meaning
Rule of zero	If possible, use standard library members so you define no custom destructor/copy/move.
Rule of three	If you define destructor, copy constructor, or copy assignment, you likely need all three.
Rule of five	In C++11, also consider move constructor and move assignment.

11. Move Semantics and Value Categories

Why move exists

Before C++11, returning or passing large objects often meant copying. C++11 introduced move semantics, allowing an object to transfer resources instead of duplicating them. A moved-from object remains valid but its value is usually unspecified unless the type documents more.

lvalues and rvalues

Expression kind	Simple explanation	Example
lvalue	Has identity; you can usually take its address	x
rvalue	Temporary or expiring value	42, std::string("x")
xvalue	Expiring value, often created by std::move	std::move(s)

std::move

```
std::string a = "large text";
std::string b = std::move(a); // a may now be empty, but is still valid

// You may assign a new value to a moved-from object:
a = "new value";
```

std::move does not move by itself. It casts an expression to an rvalue, allowing a move constructor or move assignment to be selected.

Perfect forwarding overview

```
template <class T>
void wrapper(T&& value)
{
    target(std::forward<T>(value));
}
```

In a template, T&& can be a forwarding reference. Use std::forward to preserve whether the original argument was an lvalue or rvalue.

12. Smart Pointers and Ownership

Ownership vocabulary

Term	Meaning
owning pointer	Responsible for deleting or releasing the object.
non-owning pointer/reference	Observes an object owned elsewhere.
unique ownership	Exactly one owner at a time.
shared ownership	Object destroyed when the last owner disappears.

Term	Meaning
weak reference	Observes shared object without keeping it alive.

C++11 smart pointer summary

Type	Ownership	Typical use
<code>std::unique_ptr<T></code>	Unique	Default owning pointer for heap objects.
<code>std::shared_ptr<T></code>	Shared	Use when multiple owners are truly needed.
<code>std::weak_ptr<T></code>	Non-owning observer of shared object	Break cycles; test whether object still exists.

```
std::unique_ptr<Device> dev(new Device("A1"));
dev->start();

std::shared_ptr<Device> shared(new Device("B1"));
std::weak_ptr<Device> weak = shared;

if (auto locked = weak.lock()) {
    locked->start();
}
```

C++14 added `std::make_unique`, but in pure C++11 you normally construct `unique_ptr` with `new` or write your own small helper. C++11 does have `std::make_shared`.

shared_ptr cycle edge case

```
struct Node {
    std::shared_ptr<Node> next;
    std::weak_ptr<Node> parent; // weak avoids ownership cycle
};
```

If two objects hold `shared_ptr`s to each other, their reference counts may never reach zero. Use `weak_ptr` for back-references.

13. Lambdas, Function Objects, and Callbacks

Lambda syntax

```
[capture](parameters) -> return_type {
    body
}

auto square = [](int x) { return x * x; };
std::cout << square(5); // 25
```

Capture modes

Capture	Meaning	Warning
<code>[]</code>	Capture nothing	Safest for stateless callbacks.
<code>[x]</code>	Capture x by value	Uses a copy made when lambda is created.
<code>[&x]</code>	Capture x by reference	Danger if lambda outlives x.
<code>[=]</code>	Capture used local variables by value	Can hide expensive copies.
<code>[&]</code>	Capture used local variables by reference	Can create dangling references.
<code>[this]</code>	Capture this pointer	Object must outlive lambda.

Lambda with STL algorithm

```
std::vector<int> values{1, 2, 3, 4, 5};

values.erase(std::remove_if(values.begin(), values.end(),
    [](int x) { return x % 2 == 0; }),
    values.end());
```

The erase-remove idiom is needed because `std::remove_if` rearranges elements and returns a new logical end; it does not physically shrink the container.

std::function

```
std::function<int(int)> operation;
operation = [](int x) { return x + 1; };
std::cout << operation(10);
```

Use `std::function` when you need to store any callable with a specific signature. It is flexible but may have overhead. For templates, accepting the callable as a template parameter can be faster.

14. Templates and Generic Programming

Function template

```
template <class T>
T max_value(T a, T b)
{
    return (a < b) ? b : a;
}
```

Class template

```
template <class T>
class Box {
public:
    explicit Box(T value) : value_(std::move(value)) {}
    const T& get() const { return value_; }
private:
    T value_;
};
```

Alias templates

```
template <class T>
using Vec = std::vector<T>;

Vec<int> numbers{1, 2, 3};
```

Alias templates are clearer than old typedef patterns for template aliases.

Variadic templates

```
void print_all() {}

template <class T, class... Rest>
void print_all(const T& first, const Rest&... rest)
{
    std::cout << first << '\n';
    print_all(rest...);
}
```

Variadic templates accept a variable number of template arguments. They are fundamental for modern generic libraries, forwarding helpers, and tuple-like code.

15. Inheritance, Polymorphism, override, and final

Basic inheritance

```
class Animal {
public:
    virtual ~Animal() = default;
    virtual void speak() const = 0;
};

class Dog : public Animal {
public:
    void speak() const override { std::cout << "woof\n"; }
};
```

If a class is meant to be used polymorphically through a base pointer or reference, give the base class a virtual destructor.

override and final

Keyword	Purpose
override	Asks compiler to verify that the function overrides a virtual base function.
final	Prevents further overriding of a virtual function or inheritance from a class.

```
class Base {
public:
    virtual void run(int) {}
};

class Derived : public Base {
public:
    void run(int) override {}    // compiler checks signature
};
```

Use override on every overriding function. It catches spelling mistakes, missing const, and wrong parameter types.

16. enum class and Stronger Type Safety

Old enum vs enum class

```
enum OldState { Idle, Running };    // names leak into surrounding scope

enum class State { Idle, Running }; // scoped and strongly typed

State s = State::Idle;
```

enum class avoids accidental conversion to int and avoids name collisions.

Underlying type

```
enum class ErrorCode : unsigned char {
    Ok = 0,
    Timeout = 1,
    Invalid = 2
};
```

Specifying the underlying type is useful for binary protocols, embedded systems, and storage-sensitive code.

17. STL Algorithms and Iterators

Iterator vocabulary

Term	Meaning
begin	Iterator pointing to first element.
end	Iterator one past the last element; not dereferenceable.
range	Pair of iterators [begin, end).
invalidated iterator	Iterator that no longer safely refers to an element after container modification.

Common algorithms

Algorithm	Purpose	Example idea
<code>std::find</code>	Find first equal value	<code>find(v.begin(), v.end(), 5)</code>
<code>std::count_if</code>	Count elements satisfying predicate	count positives
<code>std::sort</code>	Sort range	sort names alphabetically
<code>std::transform</code>	Map values from one range to another	convert Celsius to Fahrenheit
<code>std::accumulate</code>	Reduce to a single value	sum values
<code>std::remove_if</code>	Move unwanted values to end	combine with erase

```
std::vector<int> v{5, 1, 4, 2, 3};
std::sort(v.begin(), v.end());

auto pos = std::find(v.begin(), v.end(), 4);
if (pos != v.end()) {
    std::cout << "found\n";
}

int sum = std::accumulate(v.begin(), v.end(), 0);
```

Iterator invalidation edge case

```
std::vector<int> v{1, 2, 3};
auto it = v.begin();
v.push_back(4); // may reallocate, invalidating it
// *it is now unsafe
```

After modifying a container, check its invalidation rules before reusing old iterators, pointers, or references to elements.

18. Exceptions and Error Handling

try, throw, catch

```
try {
    if (!connected) {
        throw std::runtime_error("not connected");
    }
} catch (const std::exception& ex) {
    std::cerr << ex.what() << '\n';
}
```

Catch exceptions by const reference to avoid slicing and unnecessary copies.

noexcept

```
void cleanup() noexcept
{
    // must not throw
}
```

noexcept documents and enforces that a function will not throw. It can also affect optimization and move behavior in containers.

Exception safety levels

Level	Meaning
No guarantee	If an exception occurs, object/program state may be corrupted.
Basic guarantee	No leaks; invariants preserved, but value may change.
Strong guarantee	Operation succeeds or has no effect.
No-throw guarantee	Operation never throws.

19. Concurrency in C++11

std::thread

```
void worker(int id)
{
    std::cout << "worker " << id << '\n';
}

std::thread t(worker, 1);
t.join(); // wait for thread to finish
```

A joinable `std::thread` must be joined or detached before destruction. Destroying a joinable thread calls `std::terminate`.

mutex and lock_guard

```
std::mutex m;
int counter = 0;

void increment()
{
    std::lock_guard<std::mutex> lock(m);
    ++counter;
}
```

`lock_guard` is RAII for a mutex: it locks in the constructor and unlocks in the destructor.

future and async

```
auto fut = std::async(std::launch::async, [] {
    return 42;
});

int result = fut.get();
```

A future represents a result that may become available later. Calling `get()` waits and retrieves it once.

atomic

```
std::atomic<bool> stop{false};

void request_stop()
{
    stop.store(true);
}
```

Use atomics for simple shared flags and counters. For compound invariants involving multiple variables, use a mutex.

20. Chrono, Random, Tuple, Regex, and Utility Types

chrono

```
using clock = std::chrono::steady_clock;

auto start = clock::now();
// work...
auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(
    clock::now() - start);
```

Use `steady_clock` for measuring elapsed time because it is monotonic. Use `system_clock` for wall-clock time.

random

```
std::mt19937 rng{std::random_device{}()};
std::uniform_int_distribution<int> dist(1, 6);
int dice = dist(rng);
```

Prefer `<random>` over `rand()`. Separate the engine from the distribution.

tuple and pair

```
std::pair<int, std::string> p{1, "one"};
std::tuple<int, double, std::string> t{1, 3.14, "pi"};

int id = std::get<0>(t);
```

regex

```
std::regex pattern(R"(\d{4}-\d{2}-\d{2})");
std::string text = "2026-05-20";

if (std::regex_match(text, pattern)) {
    std::cout << "date-like string\n";
}
```

C++11 regex support exists in the standard library, but performance and completeness varied in older compilers. Test on your target compiler.

21. Preprocessor, Headers, and Compilation Hygiene

Include guards

```
#ifndef DEVICE_H
#define DEVICE_H

class Device {};

#endif // DEVICE_H
```

Include guards prevent duplicate declarations when a header is included more than once. `#pragma once` is common but not formally standardized in C++11.

Macros

```
#define BUFFER_SIZE 128

constexpr int better_buffer_size = 128;
```

Prefer `constexpr`, `const`, inline functions, and templates over macros when possible. Macros are textual substitution and ignore scope and type rules.

Forward declarations

```
class Device; // forward declaration

void use(Device& d);
```

Forward declarations reduce compile-time dependencies. You need the full definition when using object size, members, inheritance, or deleting through a pointer.

22. C++11 Cheat Sheets

Keywords and feature reminders

Feature	Syntax	Remember
<code>auto</code>	<code>auto x = expr;</code>	Compile-time type deduction.
<code>decltype</code>	<code>decltype(expr)</code>	Type of an expression; parentheses matter.
<code>nullptr</code>	<code>int* p = nullptr;</code>	Type-safe null pointer.
range for	<code>for (auto& x : c)</code>	Works with <code>begin/end</code> .
<code>lambda</code>	<code>[=](int x){...}</code>	Capture controls lifetime risk.
<code>enum class</code>	<code>enum class State { A };</code>	Scoped, no implicit int conversion.
<code>override</code>	<code>void f() override;</code>	Compiler checks overriding.
<code>final</code>	<code>class X final</code>	Prevents further inheritance/override.
<code>constexpr</code>	<code>constexpr int n = 4;</code>	Potential compile-time evaluation.
<code>static_assert</code>	<code>static_assert(cond, msg);</code>	Compile-time check.
<code>noexcept</code>	<code>void f() noexcept;</code>	Promises no exceptions.

STL headers quick reference

Header	Contains
<code><vector></code>	<code>std::vector</code>
<code><array></code>	<code>std::array</code>
<code><map></code> , <code><unordered_map></code>	Associative maps
<code><memory></code>	<code>unique_ptr</code> , <code>shared_ptr</code> , <code>weak_ptr</code>

Header	Contains
<algorithm>	sort, find, transform, remove_if
<numeric>	accumulate
<functional>	function, bind, function objects
<thread>, <mutex>, <future>, <atomic>	Concurrency
<chrono>	Time durations and clocks
<random>	Random engines and distributions
<tuple>	tuple, get
<regex>	Regular expressions

Common compiler diagnostics vocabulary

Diagnostic phrase	Likely meaning
no matching function	Overload resolution found no callable function for your argument types.
undefined reference	Compilation succeeded, but linker cannot find a definition.
use of deleted function	You tried to copy/move/construct something that is explicitly deleted or implicitly unavailable.
cannot bind non-const lvalue reference to rvalue	A function expects T&, but you passed a temporary.
invalid conversion	Compiler refuses unsafe type conversion.
ambiguous overload	More than one overload is equally good.
multiple definition	Same non-inline symbol defined in more than one translation unit.

23. Pitfalls and Edge Cases

Most common C++11 mistakes

Mistake	Better habit
Returning pointer/reference to a local variable	Return by value or use ownership-aware objects.
Using raw new/delete in application code	Prefer containers and smart pointers.
Capturing local variables by reference in long-lived lambdas	Capture by value or ensure lifetime.
Forgetting virtual destructor in polymorphic base class	Add virtual ~Base() = default.
Ignoring iterator invalidation	Reacquire iterators after modifying containers.
Using shared_ptr everywhere	Use unique_ptr by default; shared ownership only when needed.
Thinking std::move moves	It only casts; moving happens when a move operation is selected.
Assuming moved-from object has a useful value	Only rely on it being valid/destructible/assignable unless documented.
Putting using namespace std in headers	Use std:: prefixes or local using declarations in .cpp files.

Undefined behavior examples

```
int* p = nullptr;
// *p = 1;           // UB: dereference null

int a[3]{1,2,3};
// int x = a[3];     // UB: out-of-bounds

int i = 2147483647;
// ++i;             // UB for signed int overflow
```

Undefined behavior is not a runtime exception. The compiler may optimize assuming it never happens, so bugs can look random or impossible.

24. Mini Practice Lab

Exercise 1: Replace raw ownership

Rewrite this code using `std::unique_ptr` or stack allocation.

```
Device* d = new Device("A1");
d->start();
delete d;
```

Exercise 2: Use algorithms

Given a vector of integers, remove all negative values and compute the sum of the remaining values.

```
std::vector<int> values{10, -2, 5, -7, 3};
```

Exercise 3: Write a safe callback

Create a lambda callback that prints a device id. Decide whether to capture by value or reference, and explain the lifetime consequence.

Exercise 4: Thread safety

Two threads increment the same counter. Protect the counter using a mutex, or use an atomic integer if the operation is only a simple counter increment.

25. Compact Glossary

Word	Definition
ABI	Application Binary Interface; binary-level compatibility contract.
ADL	Argument-dependent lookup; namespaces associated with argument types participate in lookup.
aggregate	Simple type initialized directly from its public members.
copy elision	Compiler omits a copy/move in certain object creation cases.
dynamic storage	Storage obtained from free store, usually with <code>new</code> or allocators.
free store	C++ memory area used by <code>new/delete</code> .
invariant	Condition that must remain true for an object to be valid.
ODR	One Definition Rule; entities must have exactly the allowed number of definitions.
POD	Plain Old Data; C++11 refines this into more precise type traits.
RAII	Manage resources with object lifetime.
SFINAE	Substitution Failure Is Not An Error; template substitution failures can remove overloads.

Word	Definition
translation unit	Source file after preprocessing.
type erasure	Hide a concrete type behind a uniform interface, e.g., <code>std::function</code> .
value category	Classification of expressions such as lvalue, prvalue, xvalue.

26. Final One-Page Review

When reading or writing C++11, repeatedly ask four questions:

- Who owns this object or resource? Prefer RAII, containers, and smart pointers.
- What is the lifetime of the object referenced by this pointer, reference, iterator, or lambda capture?
- Is this expression copied, moved, referenced, or deduced by value?
- Can the standard library algorithm or container express this more safely than manual code?

The modern C++11 style is not about using every new feature. It is about writing code where ownership, lifetime, type intent, and error handling are clear to both the compiler and the human reader.